La notion de décorateur en Python

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Introduction aux décorateurs **3**

A. Introduction aux décorateurs 3

B. Exercice : Quiz 6

III. Particularités des décorateurs **7**

A. Particularités des décorateurs 7

B. Exercice : Quiz 9

IV. L'essentiel **10**

V. Auto-évaluation **11**

A. Exercice 11

B. Test 11

Solutions des exercices **12**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée (en minutes) : 60

Environnement de travail : Python 3.12+, PyCharm Community

Pré requis : bases en Python

Contexte

Une des fonctionnalités qui distinguent Python est le concept de décorateur.

Les décorateurs sont des outils qui permettent d'ajouter ou de modifier le comportement des fonctions ou des classes sans changer leur contenu. Les fonctions continuent ainsi à pouvoir être utilisées normalement.

Dans ce cours, nous allons explorer en profondeur les décorateurs, comprendre comment ils fonctionnent et découvrir les avantages qu'ils offrent.

Nous verrons que le langage Python met à notre disposition toute une série de décorateurs standards qui permettent de couvrir énormément de cas d'utilisation.

Grâce à une compréhension approfondie des notions de décorateurs, vous rendrez votre code Python plus propre, plus modulaire et plus efficace.

II. Introduction aux décorateurs

A. Introduction aux décorateurs

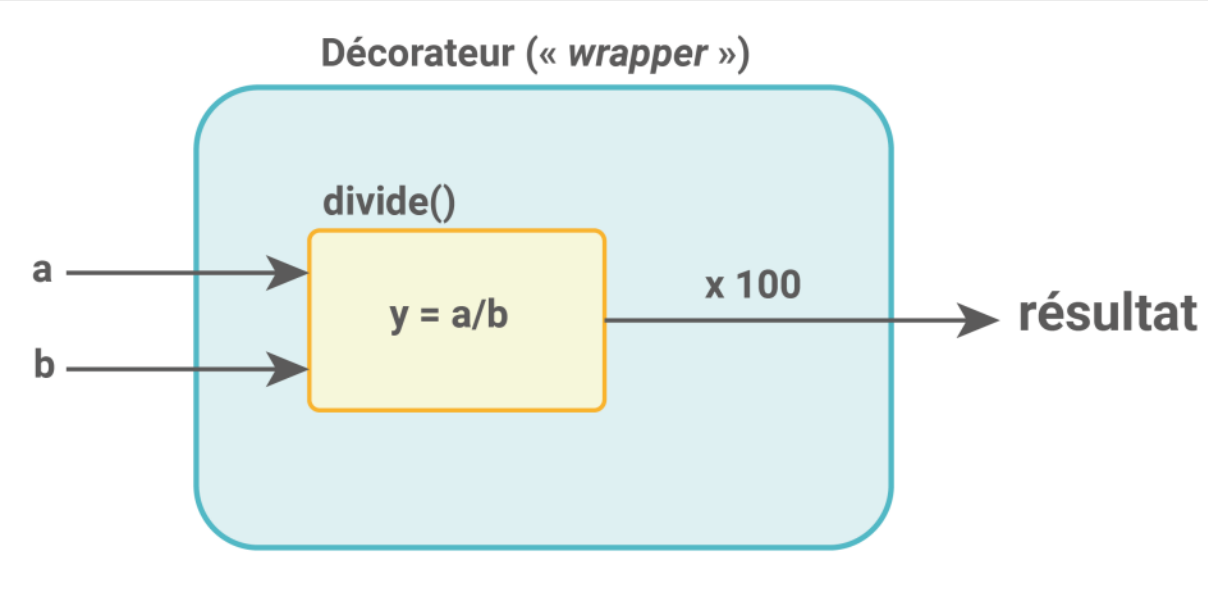
Définition **Qu'est ce qu'un décorateur ?**

Un décorateur (parfois appelé « *wrapper* », qui signifie « *emballage* ») est littéralement un bloc de code (ou même une fonction) qui exécute une fonction donnée en l'encapsulant dans un code spécifique. De cette manière, les décorateurs permettent de modifier la manière dont une fonction est appelée sans faire la moindre modification dans la fonction elle-même. On peut ainsi par exemple modifier la valeur des arguments avec laquelle elle est appelée, ou modifier sa valeur de retour. On peut ainsi créer des « *variantes* » d'une même fonction, qui seraient utilisées par plusieurs objets différents.

Exemple **Exemple de décorateur**

Imaginons que nous souhaitons calculer le pourcentage d'utilisation du serveur qui héberge notre site web. Ce pourcentage est donné par le nombre d'utilisateurs actuellement connectés, divisé par sa capacité, et le tout multiplié par 100.

On peut définir une fonction qui calcule le résultat de la division, et utiliser un décorateur afin de représenter le ratio en pourcents.



Les cas d'utilisation

Les décorateurs peuvent être employés dans des situations très diverses. Cela peut aller de la simple volonté de vouloir « *diversifier* » une fonction à des évaluations de performance, ou encore de debugging. Par exemple, on pourrait imaginer vouloir calculer le temps d'exécution d'une fonction donnée. On pourrait alors définir un décorateur qui démarre un chronomètre avant l'appel à la fonction, et qui arrêterait celui-ci dès la fin d'exécution de la fonction.

L'utilisation des décorateurs dans les frameworks de développement web, tels que Django, sont très fréquents. Par exemple, il existe des décorateurs qui permettent d'afficher une page web en appliquant certaines restrictions, dépendantes du profil de l'utilisateur qui visionne la page. Dans ce cas de figure, c'est « *l'emballage* » de la fonction d'affichage (c'est-à-dire le contenu du décorateur) qui procédera aux vérifications, avant d'afficher la page d'une façon ou d'une autre.

Analyse de la structure d'un décorateur

Un décorateur prend une fonction en argument. Il définit une nouvelle fonction, qui altère éventuellement la logique de la fonction initiale. Ensuite, le décorateur renvoie (« *return* ») la fonction résultante.

Pour reprendre l'exemple précédent annoté :

```
1 def decorate(func):       # Definition du décorateur
2     def wrapper(*args):     # Définition de la function wrapper
3         y = func(*args)
4         result = y*100
5         print(result)
6         return result
7     return wrapper       # Le décorateur renvoie le wrapper
8
9
10 @decorate         # Spécification du décorateur
11 def divide(a, b):
12     return a/b
13
14
15 divide(10,100)
```


Tous droits réservés à STUDI - Reproduction interdite

Dans ce cas, @decorate est notre décorateur. C'est comparable à ce qu'on peut appeler du « *sucre syntaxique* », c'est-à-dire une simplification d'écriture. Dans cet exemple, l'appel à divide(10,100) renvoie en réalité decorate(divide(10,100)).

Remarque **Décorateur à usage général**

Dans l'exemple précédent, le 'wrapper', c'est-à-dire la fonction utilisée dans le décorateur decorate() prend un nombre indéfini d'arguments. Ceci est spécifié par l'astérisque « * ». On aurait également pu spécifier le double astérisque « ** » mentionnant que notre fonction de départ (func()) prend en charge les arguments de type « *clé-valeur* ». On aurait donc eu la signature suivante pour le wrapper :

```
def wrapper(*args, **kwargs):  # args = arguments, kwargs = arguments clé-valeur (keyword-arguments)
```

Ce type de décorateur est appelé décorateur à usage général (« *general-purpose decorator* »), car il est capable d'encapsuler n'importe quel type de fonction func(), peu importe le nombre d'arguments qu'elle prend en charge dans son implémentation.

Complément **Quelques décorateurs natifs de Python**

Python implémente nativement certains décorateurs, qui permettent donc de définir un cadre pour l'exécution de certaines méthodes ou attributs. Le tableau suivant synthétise quelques décorateurs communs, dont l'utilisation est très fréquente :

Décorateur	Description
@property	Permet de définir des getters et des setters pour les attributs d'une classe. Il permet d'encapsuler la logique d'accès aux attributs en évitant l'accès direct aux attributs.
@staticmethod	Un décorateur utilisé pour définir une méthode statique dans une classe ce qui signifie que la méthode n'est pas liée à une instance spécifique de la classe et ne peut donc pas modifier l'état d'une instance de cette classe.
@classmethod	Utilisé pour définir une méthode de classe, qui est une méthode qui peut être appelée sur la classe elle-même plutôt que sur une instance de la classe. La méthode de classe prend comme premier paramètre la classe elle-même (habituellement nommée « *cls* »).
@abstractmethod	Permet de définir une méthode abstraite, c'est-à-dire que l'implémentation de la fonction n'est pas déterminée. On peut par exemple définir une méthode abstraite dans une classe parent, en spécifiant uniquement son nom et ses arguments, mais en définir l'implémentation dans les classes enfants. Attention, ce décorateur nécessite le package abc (import abc)
@atexit.register	Ce décorateur permet de définir des fonctions qui seront exécutés systématiquement à la terminaison du programme. Attention, ce décorateur nécessite le package atexit (import atexit)

Tous droits réservés à STUDI - Reproduction interdite

Décorateur	Description
@typing.final	Permet d'empêcher la surcharge d'une fonction. Si une classe enfant surcharge une méthode de sa classe parent, l'IDE le signalera. Attention, ce décorateur nécessite le package typing (import typing)

B. Exercice : Quiz

[solution n°1 p.13]

Question 1

Qu'est-ce qu'un décorateur ?

- Une fonction qui modifie le comportement d'une autre fonction sans changer son code source
- Un outil de mise en forme du code pour le rendre plus lisible
- Un système de commentaire pour expliquer le code
- Une façon de créer des graphiques en Python

Question 2

Qu'est ce qu'un « *wrapper* » ?

- Un mot clé du langage Python qui permet de définir un décorateur
- Une variable qui porte ce nom de manière arbitraire, utilisée dans le cadre des décorateurs
- Une fonction qui porte ce nom de manière arbitraire, utilisée dans le cadre des décorateurs
- Un mot clé du langage Python qui permet de définir le décorateur @wrapper

Question 3

Comment déclare-t-on un décorateur ?

- En utilisant le symbole @ avant le nom du décorateur
- En utilisant le symbole $ avant le nom du décorateur
- En mettant le nom du décorateur entre guillemets
- En utilisant le mot clé « *decorator* » suivi du nom du décorateur

Question 4

Qu'est-ce qu'un décorateur à usage général (general-purpose decorator) ?

- Un décorateur qui porte le nom @generalpurpose
- Un décorateur qui s'applique a des classes ainsi qu'à des fonctions
- Un décorateur qui peut s'appliquer à des fonctions avec un nombre indéfini de paramètres d'entrée
- Un décorateur dont le comportement n'est pas défini

Question 5

Tous droits réservés à STUDI - Reproduction interdite

Existe-t-il des décorateurs définis nativement dans le langage Python ?

- Non, les décorateurs doivent être définis par l'utilisateur
- Oui, mais uniquement pour les décorateurs de classe
- Oui, mais ils ne sont jamais employés
- Oui, et ils sont très souvent employés

III. Particularités des décorateurs

A. Particularités des décorateurs

Définition **Décorateurs et paramètres**

L'ajout de paramètres à un décorateur nécessite une modification de la structure du décorateur. En plus de la fonction de haut niveau qui prend la fonction à modifier en argument, on a besoin d'une autre fonction de haut niveau qui prendra les paramètres du décorateur et qui renverra le décorateur lui-même.

Exemple **Décorateur avec paramètres**

```
1 def decorate_with_params(message_before, message_after):
2     def decorate(func):
3         def wrapper():
4             print(message_before)
5             func()
6             print(message_after)
7         return wrapper
8     return decorate
9
10 @decorate_with_params("Before", "After")
11 def hello():
12     print("Hello!")
```

Dans cet exemple, decorate_with_params() est une fonction de haut niveau qui prend deux paramètres (message_before et message_after) et renvoie un décorateur (decorate). Ce décorateur modifie l'environnement dans lequel l'appel à la fonction hello() est fait en y ajoutant deux messages à l'écran.

Définition **Décorateurs multiples**

Il est possible d'utiliser plusieurs décorateurs sur une seule fonction. Lorsque plusieurs décorateurs sont appliqués à une fonction, ils sont exécutés dans l'ordre de proximité à la fonction. Autrement dit, le décorateur le plus proche de la fonction (celui écrit juste au-dessus) est appliqué en premier, suivi du décorateur au-dessus de lui, et ainsi de suite.

Chaque décorateur reçoit la fonction (ou le résultat du décorateur précédent) et retourne une nouvelle fonction modifiée. Ainsi, une série de transformations est appliquée à la fonction originale, chacune correspondant à l'action d'un décorateur.

Exemple **Décorateurs multiples**

```
1 def decorate_outer(func):
2     def wrapper():
3         print("begin outer decorate")
4         func()
5         print("end outer decorate")
6     return wrapper
7
```

Tous droits réservés à STUDI - Reproduction interdite

```
8 def decorate_inner(func):
9     def wrapper():
10         print("begin inner decorate")
11         func()
12         print("end inner decorate")
13     return wrapper
14
15 @decorate_outer
16 @decorate_inner
17 def MyFun():
18     print("Initial function")
19
20 MyFun()
```

Dans cet exemple, on constate qu'on fait précéder la définition de MyFun() de deux décorateurs. La fonction est d'abord exécutée dans le décorateur intérieur (decorate_inner), qui est spécifié à la ligne juste au dessus la déclaration de la fonction initiale. Ensuite, le décorateur extérieur (decorate_outer) est appliqué sur la sortie du premier décorateur.

Définition **Décorateur de classe**

Python étant un langage orienté objet, il prend en charge non seulement les décorateurs de fonctions, mais aussi les décorateurs de classes. Les décorateurs de classes, comme leur nom l'indique, sont appliqués aux classes plutôt qu'aux fonctions. Un décorateur de classe est une fonction qui prend une classe en tant qu'argument, la modifie ou l'étend d'une certaine manière, puis la renvoie.

De la même manière qu'un décorateur de fonction peut modifier le comportement d'une fonction, un décorateur de classe peut modifier le comportement d'une classe. Cela signifie qu'il peut modifier les méthodes de la classe, ajouter de nouvelles méthodes, ou même changer les attributs de la classe.

Les décorateurs de classes sont plus rares que les décorateurs de fonctions, et sont généralement utilisés pour des cas d'utilisation plus avancés.

Attention

Les décorateurs de classe sont à utiliser avec précaution. En effet, puisqu'ils modifient le comportement de la classe, ils peuvent rendre le code plus difficile à comprendre s'ils sont utilisés à l'excès ou de manière inappropriée.

Exemple **Décorateur de classe**

```
1 def decorate_class(cls):
2     class ClassWrapper(cls):
3         def MyMethod(self):
4             return "My decorated method implementation"
5     return ClassWrapper
6
7 @decorate_class
8 class InitialClass:
9     def MyMethod(self):
10         return "My method implementation"
11
12
13 obj = InitialClass()
14 print(obj.MyMethod())  # Imprime "My decorated method implementation"
```


Tous droits réservés à STUDI - Reproduction interdite

Dans cet exemple, on définit un décorateur de classe (decorate_class) qui contient un wrapper (ClassWrapper). Ce wrapper prend en argument une classe (cls), et définit une méthode MyMethod() légèrement différente de l'implémentation originale. Le résultat de l'appel à obj.MyMethod() est bien conforme à la définition de notre décorateur de classe.

Bonnes pratiques et pièges à éviter

Lorsque vous utilisez des décorateurs en Python, il est important de faire attention à certains pièges courants. Par exemple, l'ordre d'application des décorateurs peut avoir un impact significatif sur le comportement de votre code. En règle générale, les décorateurs sont appliqués de bas en haut et de droite à gauche (il s'agit de décorateurs imbriqués).

De plus, lorsque vous utilisez des décorateurs, les métadonnées de la fonction originale, comme son nom et sa documentation, peuvent être perdues. Pour préserver ces métadonnées, vous pouvez utiliser la fonction wraps du module functools.

Exemple

```
1 from functools import wraps
2
3 def decorate(func):
4     @wraps(func)
5     def wrapper():
6         print("before")
7         func()
8         print("after")
9     return wrapper
10
11 @decorate
12 def Hello():
13     """my method documentation"""
14     print("Hello!")
15
16 print(Hello.__name__)     # Affiche : Hello
17 print(Hello.__doc__)      # Affiche : my method documentation
```

Dans cet exemple, malgré l'application du décorateur, la fonction Hello() conserve son nom original et sa « *docstring* » grâce à l'utilisation de @wraps(func). C'est une bonne pratique à suivre pour s'assurer que les métadonnées de votre fonction restent intactes lors de l'application des décorateurs.

B. Exercice : Quiz

[solution n°2 p.14]

Question 1

Comment ajouter des paramètres à un décorateur ?

- ○ En ajoutant des arguments à la fonction de haut niveau du décorateur
- ○ En ajoutant une autre fonction de haut niveau qui prend les paramètres du décorateur
- ○ En utilisant le mot-clé « *param* » suivi des paramètres
- ○ En utilisant le mot-clé « *args* » suivi des paramètres

Question 2

Tous droits réservés à STUDI - Reproduction interdite

Que fait un décorateur de classe ?

- ○ Il modifie le comportement d'une classe, comme modifier les méthodes de la classe, ajouter de nouvelles méthodes, ou changer les attributs de la classe
- ○ Il crée une nouvelle classe basée sur une classe existante
- ○ Il change le nom d'une classe existante
- ○ Il crée une instance d'une classe existante

Question 3

Quel est l'impact de l'ordre d'application des décorateurs multiples ?

- ○ L'ordre d'application n'a aucun impact sur le comportement du code
- ○ L'ordre d'application peut changer le comportement du code
- ○ L'ordre d'application est aléatoire d'une exécution à l'autre
- ○ L'ordre d'application est toujours du haut vers le bas

Question 4

Comment préserver les métadonnées d'une fonction lorsqu'elle est décorée en Python ?

- ○ En utilisant la fonction « *preserve* » du module « *metadata* »
- ○ En utilisant la fonction « *save* » du module « *metadata* »
- ○ En utilisant la fonction « *wraps* » du module « *functools* »
- ○ En utilisant la fonction « *save* » du module « *functools* »

Question 5

À quoi faut-il faire attention lors de l'utilisation des décorateurs ?

- ☐ A la modification du comportement des fonctions ou des classes lorsqu'elles sont associées à un décorateur
- ☐ Il n'y a pas de point d'attention spécifique, les décorateurs ne modifient pas le comportement des fonctions ou des classes
- ☐ A l'ordre d'application des décorateurs lors de l'utilisation de décorateurs multiples
- ☐ Au nom du décorateur qui doit être défini dans la syntaxe Python

IV. L'essentiel

Un décorateur est une fonction qui prend une autre fonction en argument, modifie ou étend son comportement, puis la renvoie, le tout sans altérer son code source initial ni sa définition.

Par ailleurs, les décorateurs ne se limitent pas aux fonctions : en effet, Python étant un langage orienté objet, il prend en charge les décorateurs de classes, permettant de modifier ou d'étendre le comportement des classes. Qu'il s'agisse d'ajouter de nouvelles méthodes, de modifier les méthodes existantes ou de changer les attributs d'une classe, les décorateurs de classe offrent une puissante flexibilité dans la gestion de vos objets.

Enfin, l'utilisation des décorateurs nécessite une certaine attention. L'ordre d'application des décorateurs peut significativement influencer le comportement de votre code. De même, lors de l'utilisation de décorateurs, les métadonnées de la fonction originale peuvent être perdues. Ainsi, il est essentiel d'apprendre à les préserver en utilisant des outils tels que la fonction « *wraps* » du module « *functools* ».

Tous droits réservés à STUDI - Reproduction interdite

Les décorateurs sont un outil qui est largement utilisé en Python, dans tous les domaines d'applications, mais surtout dans le domaine du développement Web. Les frameworks Python définissent généralement leurs propres décorateurs afin de simplifier leur utilisation.

V. Auto-évaluation

A. Exercice

Question 1

[solution n°3 p.15]

Écrivez une fonction simple "send_message()" qui prend un message et l'envoie à un utilisateur. Pour le moment, nous supposerons que l'envoi d'un message consiste simplement à imprimer le message et l'utilisateur à l'écran.

Question 2

[solution n°4 p.15]

Écrivez un décorateur "chronometer" qui mesure le temps d'exécution d'une fonction. Appliquez ce décorateur à la fonction "send_message()". Vous pouvez utiliser la librairie "time" (import time).

Question 3

[solution n°5 p.16]

Écrivez un autre décorateur "logging" qui enregistre chaque appel à la fonction et les arguments utilisés. Appliquez ce décorateur à la fonction "send_message()".

B. Test

Exercice 1 : Quiz

[solution n°6 p.16]

Question 1

Pourquoi dit-on que l'utilisation de décorateurs respecte l'utilisation du principe « *DRY* » (« *Don't Repeat Yourself* »), selon lequel le code est plus facilement lisible et moins long ?

- ○ Parce qu'ils permettent d'éviter la répétition de certaines parties du code
- ○ Parce qu'ils permettent d'écrire des boucles plus efficacement
- ○ Parce qu'ils accélèrent l'exécution du code
- ○ Parce qu'ils assurent la correction automatique des erreurs

Question 2

Quelle est la différence entre un décorateur de fonction et un décorateur de classe ?

- ○ Un décorateur de fonction est appliqué sur une fonction, en modifiant son comportement. Un décorateur de classe est appliqué sur une classe, et peut altérer ses méthodes ou attributs
- ○ Les deux types de décorateurs ne diffèrent que par leur nom
- ○ Un décorateur de classe peut être appliqué sur une fonction, mais pas l'inverse
- ○ Les décorateurs de fonctions et de classes ne servent qu'à rendre le code plus lisible et n'affectent pas le comportement du programme

Question 3

Tous droits réservés à STUDI - Reproduction interdite

Quelle est la fonction du symbole « @ » en Python lorsqu'il est utilisé avec les décorateurs ?

- Il signifie que le code qui suit doit être ignoré
- Il est utilisé pour créer une boucle dans le code Python
- Il indique que le code qui suit est un décorateur
- Il est utilisé pour marquer une section de code comme obsolète

Question 4

A quoi sert le décorateur @staticmethod ?

- A définir une méthode statique, c'est-à-dire qu'elle est identique pour toutes les instances d'une même classe
- A définir une méthode qui ne renvoie aucun résultat
- A définir une méthode qui ne prend aucun argument en entrée

Question 5

L'utilisation des décorateurs est obligatoire lors du développement Python…

- Non, tout ce qui est faisable avec des décorateurs peut être fait autrement. Cependant les décorateurs facilitent l'écriture et la conception
- Oui, certaines manipulation de fonctions ou de classes requièrent l'utilisation de décorateurs
- Oui, sinon le code ne peut pas s'exécuter
- Non, car les décorateurs ne peuvent pas modifier le comportement du code

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

studi

Exercice p. 6 Solution n°1

Question 1

Qu'est-ce qu'un décorateur ?

- ⦿ Une fonction qui modifie le comportement d'une autre fonction sans changer son code source
- ○ Un outil de mise en forme du code pour le rendre plus lisible
- ○ Un système de commentaire pour expliquer le code
- ○ Une façon de créer des graphiques en Python

Q Un décorateur est une fonction spéciale qui prend une autre fonction (ou une classe) en entrée et étend ou modifie son comportement sans modifier son code source initial.

Question 2

Qu'est ce qu'un « *wrapper* » ?

- ○ Un mot clé du langage Python qui permet de définir un décorateur
- ○ Une variable qui porte ce nom de manière arbitraire, utilisée dans le cadre des décorateurs
- ⦿ Une fonction qui porte ce nom de manière arbitraire, utilisée dans le cadre des décorateurs
- ○ Un mot clé du langage Python qui permet de définir le décorateur @wrapper

Q wrapper() est généralement le nom qu'on donne à la fonction interne d'un décorateur, afin de redéfinir ou « *d'emballer* » la fonction initiale pour en modifier le comportement. Un décorateur retourne cette fonction à la fin de sa définition : « *return wrapper* ». Puisqu'il s'agit d'un nom de méthode, on pourrait très bien utiliser un autre nom, comme « *decorator_function()* » par exemple.

Question 3

Comment déclare-t-on un décorateur ?

- ⦿ En utilisant le symbole @ avant le nom du décorateur
- ○ En utilisant le symbole $ avant le nom du décorateur
- ○ En mettant le nom du décorateur entre guillemets
- ○ En utilisant le mot clé « *decorator* » suivi du nom du décorateur

Q Un décorateur est déclaré par l'utilisation du symbole « @ » suivi du nom du décorateur.

Question 4

Qu'est-ce qu'un décorateur à usage général (general-purpose decorator) ?

- ○ Un décorateur qui porte le nom @generalpurpose
- ○ Un décorateur qui s'applique a des classes ainsi qu'à des fonctions
- ⦿ Un décorateur qui peut s'appliquer à des fonctions avec un nombre indéfini de paramètres d'entrée
- ○ Un décorateur dont le comportement n'est pas défini

Tous droits réservés à STUDI - Reproduction interdite

Un décorateur à usage général est un décorateur dont le wrapper est compatible de n'importe quelle fonction, avec n'importe quel nombre de paramètres d'entrée.

Question 5

Existe-t-il des décorateurs définis nativement dans le langage Python ?

- ○ Non, les décorateurs doivent être définis par l'utilisateur
- ○ Oui, mais uniquement pour les décorateurs de classe
- ○ Oui, mais ils ne sont jamais employés
- ◉ Oui, et ils sont très souvent employés

Python définit pour nous plusieurs décorateurs qui permettent une simplification du code ainsi qu'une meilleure lisibilité. Ces décorateurs sont très souvent utilisés par les développeurs Python. Les frameworks Python, comme Django par exemple, définissent très souvent leurs propres décorateurs.

Exercice p. 9 Solution n°2

Question 1

Comment ajouter des paramètres à un décorateur ?

- ○ En ajoutant des arguments à la fonction de haut niveau du décorateur
- ◉ En ajoutant une autre fonction de haut niveau qui prend les paramètres du décorateur
- ○ En utilisant le mot-clé « *param* » suivi des paramètres
- ○ En utilisant le mot-clé « *args* » suivi des paramètres

Pour ajouter des paramètres à un décorateur, il faut ajouter une autre fonction de haut niveau qui prend les paramètres du décorateur et renvoie le décorateur lui-même.

Question 2

Que fait un décorateur de classe ?

- ◉ Il modifie le comportement d'une classe, comme modifier les méthodes de la classe, ajouter de nouvelles méthodes, ou changer les attributs de la classe
- ○ Il crée une nouvelle classe basée sur une classe existante
- ○ Il change le nom d'une classe existante
- ○ Il crée une instance d'une classe existante

Un décorateur de classe est une fonction qui prend une classe en tant qu'argument, modifie ou étend cette classe d'une certaine manière, puis la renvoie. Cela peut inclure la modification des méthodes existantes, l'ajout de nouvelles méthodes, ou la modification des attributs de la classe.

Question 3

Quel est l'impact de l'ordre d'application des décorateurs multiples ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ L'ordre d'application n'a aucun impact sur le comportement du code
- ◉ L'ordre d'application peut changer le comportement du code
- ○ L'ordre d'application est aléatoire d'une exécution à l'autre
- ○ L'ordre d'application est toujours du haut vers le bas

L'ordre d'application des décorateurs peut avoir un impact significatif sur le comportement du code. En général, les décorateurs sont appliqués de bas en haut et de droite à gauche.

Question 4

Comment préserver les métadonnées d'une fonction lorsqu'elle est décorée en Python ?

- ○ En utilisant la fonction « *preserve* » du module « *metadata* »
- ○ En utilisant la fonction « *save* » du module « *metadata* »
- ◉ En utilisant la fonction « *wraps* » du module « *functools* »
- ○ En utilisant la fonction « *save* » du module « *functools* »

Pour préserver les métadonnées d'une fonction lorsqu'elle est décorée, vous pouvez utiliser la fonction « *wraps* » du module « *functools* ». Cela permet entre autres de conserver les informations comme le nom de la fonction et sa documentation (docstring).

Question 5

À quoi faut-il faire attention lors de l'utilisation des décorateurs ?

- ☑ A la modification du comportement des fonctions ou des classes lorsqu'elles sont associées à un décorateur
- ☐ Il n'y a pas de point d'attention spécifique, les décorateurs ne modifient pas le comportement des fonctions ou des classes
- ☑ A l'ordre d'application des décorateurs lors de l'utilisation de décorateurs multiples
- ☐ Au nom du décorateur qui doit être défini dans la syntaxe Python

Les points d'attention sont : l'éventuelle modification du comportement des fonctions ou des classes qui sont utilisées dans le cadre d'un décorateur, ainsi que l'ordre d'application des décorateurs lorsque les décorateurs multiples sont employés. Les décorateurs portent des noms arbitraires, même si certains décorateurs sont propres au langage ou éventuellement au framework Python utilisé.

p. 11 Solution n°3

```
1 def send_message(user, message):
2     print(f'The message "{message}" is sent to {user}')
```

La fonction "send_message()" prend deux paramètres en entrée : "user" (représentant l'utilisateur) et "message" (représentant le message à envoyer). Elle utilise ensuite la fonction "print" pour afficher un message formaté qui indique que le message a été envoyé à l'utilisateur spécifié.

p. 11 Solution n°4

Tous droits réservés à STUDI - Reproduction interdite

```python
import time


def chronometer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f'The function {func.__name__} execution time is = {end - start:.2f} secs.')
        return result
    return wrapper



def send_message(user, message):
    print(f'The message {message} is sent to {user}')


send_message("Jean", "Bonjour")
```

Le décorateur « *chronometer* » prend une fonction en paramètre ("func") et définit une fonction imbriquée ("wrapper") qui mesure le temps d'exécution de la fonction fournie. Le temps de début et de fin sont enregistrés en utilisant la fonction "time.time()". Après l'exécution de la fonction, le temps total est calculé et affiché avec le nom de la fonction. Le décorateur est appliqué à la fonction "send_message()" à l'aide du décorateur « *@chronometer* ».

p. 11 Solution n°5

```python
import time


def logging(func):
    def wrapper(user, message):
        print(f'Call function "{func.__name__}" with user : "{user}" and message : "{message}"')
        return func(user, message)
    return wrapper


@logging
@chronometer
def send_message(user, message):
    print(f'The message {message} is sent to {user}')


send_message("Jean", "Bonjour")
```

Le décorateur "logging" prend une fonction en paramètre ("func") et définit une fonction imbriquée ("wrapper") qui affiche un message indiquant l'appel à la fonction avec les arguments spécifiés. Ensuite, la fonction d'origine ("func") est appelée avec les mêmes arguments. Le décorateur "logging" est appliqué à la fonction "send_message()" à l'aide du décorateur "@logging", et cette fonction est déjà décorée avec "@chronometer" pour mesurer le temps d'exécution.

Exercice p. 11 Solution n°6

Tous droits réservés à STUDI - Reproduction interdite

studi

Question 1

Pourquoi dit-on que l'utilisation de décorateurs respecte l'utilisation du principe « *DRY* » (« *Don't Repeat Yourself* »), selon lequel le code est plus facilement lisible et moins long ?

- ⦿ Parce qu'ils permettent d'éviter la répétition de certaines parties du code
- ○ Parce qu'ils permettent d'écrire des boucles plus efficacement
- ○ Parce qu'ils accélèrent l'exécution du code
- ○ Parce qu'ils assurent la correction automatique des erreurs

Q Les décorateurs aident à respecter le principe DRY (« *Don't Repeat Yourself* ») car ils permettent d'appliquer une certaine logique ou fonctionnalité à plusieurs fonctions ou classes sans avoir à répéter le même code.

Question 2

Quelle est la différence entre un décorateur de fonction et un décorateur de classe ?

- ⦿ Un décorateur de fonction est appliqué sur une fonction, en modifiant son comportement. Un décorateur de classe est appliqué sur une classe, et peut altérer ses méthodes ou attributs
- ○ Les deux types de décorateurs ne diffèrent que par leur nom
- ○ Un décorateur de classe peut être appliqué sur une fonction, mais pas l'inverse
- ○ Les décorateurs de fonctions et de classes ne servent qu'à rendre le code plus lisible et n'affectent pas le comportement du programme

Q La principale différence entre un décorateur de fonction et un décorateur de classe réside dans ce qu'ils modifient. Un décorateur de fonction modifie ou améliore le comportement d'une fonction particulière. Un décorateur de classe, quant à lui, s'applique à une classe, permettant de manipuler ses méthodes ou attributs, ou d'ajouter de nouveaux comportements à la classe.

Question 3

Quelle est la fonction du symbole « @ » en Python lorsqu'il est utilisé avec les décorateurs ?

- ○ Il signifie que le code qui suit doit être ignoré
- ○ Il est utilisé pour créer une boucle dans le code Python
- ⦿ Il indique que le code qui suit est un décorateur
- ○ Il est utilisé pour marquer une section de code comme obsolète

Q Le symbole « @ » est utilisé pour indiquer que le code qui suit est un décorateur. C'est une manière concise et claire d'appliquer un décorateur à une fonction ou une classe.

Question 4

A quoi sert le décorateur @staticmethod ?

- ⦿ A définir une méthode statique, c'est-à-dire qu'elle est identique pour toutes les instances d'une même classe
- ○ A définir une méthode qui ne renvoie aucun résultat
- ○ A définir une méthode qui ne prend aucun argument en entrée

Q Le décorateur @staticmethod est natif du langage Python. Il permet de définir une fonction statique, c'est-à-dire que la fonction sera identique pour toutes les instances d'une même classe, et ne pourra pas modifier l'état d'une instance spécifique.

Tous droits réservés à STUDI - Reproduction interdite

Question 5

L'utilisation des décorateurs est obligatoire lors du développement Python…

- ⦿ Non, tout ce qui est faisable avec des décorateurs peut être fait autrement. Cependant les décorateurs facilitent l'écriture et la conception
- ○ Oui, certaines manipulation de fonctions ou de classes requièrent l'utilisation de décorateurs
- ○ Oui, sinon le code ne peut pas s'exécuter
- ○ Non, car les décorateurs ne peuvent pas modifier le comportement du code

Les décorateurs sont un moyen simple et élégant de modifier le comportement d'une fonction ou d'une classe. Cependant, conceptuellement, il ne s'agit que d'une façon simple « *d'emballer* » une fonction ou une classe dans un « *wrapper* ». On pourrait donc définir cette fonction « *d'emballage* » directement, sans devoir recourir à cette syntaxe.

Toutefois, il faut noter que la plupart des frameworks Python, notamment en développement web, font usage récurrent des décorateurs. Il est donc important de bien maitriser leur fonctionnement pour pouvoir les utiliser à leur plein potentiel.

Tous droits réservés à STUDI - Reproduction interdite

Tous droits réservés à STUDI - Reproduction interdite